PACKET SCRAMBLER

Author: Sneha Gautam
Date: 19.08.2026

Program Description

This program performs four stages:

1. Validate the packet.
2. Scramble the packet using slicing.
3. Insert SYNC-BIT and remove zeros.
4. Check memory integrity using unpacking.

Question 1 – Input Validation and Test Data

```python
# question 1 -- input validation and test data

packet = [2,6,9,8,0,5,45,0,90,23]

if packet and len(packet) >= 10:
    print("Validation passed: Processing packet ....")
else:
    print("Validation failed : Packet is empty or too short.")

print("INITIAL PACKET", packet)
```

Question 2 – The Middle-Out Swap

```python
# question 2 -- the middle-out swap

midpoint = len(packet)//2
front_half = packet[:midpoint]
back_half = packet[midpoint:]

scrambled = back_half[::-1] + front_half
```

```python

print(id(packet) == id(front_half))

print("AFTER STAGE 2 SCRAMBLED", scrambled)
```

Question 3 – In-Place Correction

```python
# question 3 -- in place correction

middle_index = len(scrambled) // 2

if type(scrambled[middle_index]) is int:
    scrambled.insert(middle_index+1, "SYNC-BIT")

print("AFTER SYNC BIT INSERTION ", scrambled)

while 0 in scrambled:
    scrambled.remove(0)

print("FINAL SCRAMBLED AFTER ZERO REMOVAL", scrambled)
```

Question 4 – Memory Integrity Check

```python
# question 4 -- memory integrity check

first, *middle, last = scrambled

print(
    f"HEADER: {first}  "
    f"FOOTER: {last}  "
    f"BODY LENGTH: {len(middle)}"
)
```

Complete Program

```python
''' packet_scrambler
Author : Sneha Gautam
Date: 19.08.2026
```

```

This program performs four stages:

1. Validate the packet
2. Scramble the packet using slicing
3. Insert SYNC-BIT and remove zeros
4. Check memory integrity using unpacking
'''

# question 1 -- input validation and test data

packet = [2,6,9,8,0,5,45,0,90,23]

if packet and len(packet) >= 10:
    print("Validation passed: Processing packet ....")
else:
    print("Validation failed : Packet is empty or too short.")

print("INITIAL PACKET", packet)


# question 2 -- the middle-out swap

midpoint = len(packet)//2
front_half = packet[:midpoint]
back_half = packet[midpoint:]

scrambled = back_half[::-1] + front_half

print(id(packet) == id(front_half))

print("AFTER STAGE 2 SCRAMBLED", scrambled)


# question 3 -- in place correction

middle_index = len(scrambled) // 2

if type(scrambled[middle_index]) is int:
    scrambled.insert(middle_index+1, "SYNC-BIT")

```

```python
print("AFTER SYNC BIT INSERTION ", scrambled)

while 0 in scrambled:
    scrambled.remove(0)

print("FINAL SCRAMBLED AFTER ZERO REMOVAL", scrambled)


# question 4 -- memory integrity check

first, *middle, last = scrambled

print(
    f"HEADER: {first}  "
    f"FOOTER: {last}  "
    f"BODY LENGTH: {len(middle)}"
)
```